

Lab 4 Assignment Report

Student Information

Please type your name and student ID at here:

- 學生姓名: 彭靖凱
- 學號: E24134778
- 授課教授: 蔡家齊 教授
- 助教: 張峻豪

Synthesis Questions (20 pts)

Your report should demonstrate your understanding of pipeline hazards and your specific implementation strategy. Please answer the following questions:

For **each** hazard (Forwarding case, Load-Use Stall, and Control Flush), describe:

1. **Trigger Condition:** Under what specific circumstances does this hazard occur? (e.g., Which specific instruction sequence? Which register indices are compared?)
2. **Resolution Strategy:** How does your hardware resolve this hazard? (e.g., Forwarding data, inserting a bubble, or flushing the pipeline?)
3. **Implementation Logic:** How did you implement this in **Controller.v**?
 - For **Forwarding**: How do you decide priority if multiple stages have the data?
 - For **Stalls**: Which pipeline registers are held (frozen) and which are flushed?
 - For **Branches**: How do you clear the wrong-path instructions?
4. **Branch Prediction**
 - Please carefully study the branch predictor section of the handout. If it were up to you, which branch predictor design would you choose (Static vs. Dynamic, 1-bit vs. 2-bit)? Why?
 - Describe how you would design the control signals to resolve the control hazard caused by mispredictions (e.g., logic to flush instructions in IF and ID stages). (*Note: Implementation is bonus, but the conceptual description is required*).

Your Answer

1. Trigger Condition

以下說明三種 pipeline hazard (Forwarding、Load-Use Stall、Control Flush) 在本實驗中實際被偵測到的條件，並直接對應 **Controller.sv** 中的訊號與邏輯。

(A) Forwarding Hazard

Trigger Condition:

Forwarding 用來解決 **RAW (Read After Write) data hazard**。當 EX stage 的指令需要使用 **rs1** 或 **rs2**，但該暫存器的最新值尚未寫回 regfile，而是存在於後續 pipeline stage (M 或 W) 時，就會觸發 forwarding。

在本設計中，forwarding 主要發生於 **EX stage**，其判斷條件如下：

(1). EX ← M stage Forwarding

當滿足以下條件時，EX stage 的 operand 需要從 M stage 前遞：

- `is_E_use_rs1 / is_E_use_rs2`: EX 指令確實使用該來源暫存器
- `is_M_use_rd`: M stage 指令會寫回暫存器
- `E_rsX == M_rd`
- `E_rsX != 0`
- 且 M stage 指令不是 **LOAD** (`M_op != LOAD`)，因為 load 的資料尚未 ready

對應程式碼：

```
is_E_rs1_M_overlap =
    is_E_use_rs1 && is_M_use_rd &&
    (E_rs1 != 5'd0) && (E_rs1 == M_rd) &&
    (M_op != LOAD);

is_E_rs2_M_overlap =
    is_E_use_rs2 && is_M_use_rd &&
    (E_rs2 != 5'd0) && (E_rs2 == M_rd) &&
    (M_op != LOAD);
```

(2). EX ← W stage Forwarding

當 M stage 無法 forward（例如是 LOAD），但 W stage 已有正確寫回資料時，則從 W stage forward：

```
is_E_rs1_W_overlap =
    is_E_use_rs1 && is_W_use_rd &&
    (E_rs1 != 5'd0) && (E_rs1 == W_rd);

is_E_rs2_W_overlap =
    is_E_use_rs2 && is_W_use_rd &&
    (E_rs2 != 5'd0) && (E_rs2 == W_rd);
```

(B) Load-Use Stall

Trigger Condition:

當 **EX stage** 是 **LOAD** 指令，而 **ID stage** 的下一條指令立刻要使用該 **LOAD** 的目的暫存器時，就會發生 load-use hazard。

由於 load 的資料要到 **W stage** 才從 memory 取回，因此即使有 forwarding，也無法在下一個 cycle 提供資料，必須 **stall pipeline 一個 cycle**。

觸發條件如下：

- $E_op == \text{LOAD}$
- $E_rd \neq 0$
- ID stage 指令實際使用 $rs1$ 或 $rs2$
- $(D_rs1 == E_rd)$ 或 $(D_rs2 == E_rd)$

對應程式碼：

```
is_load_use_stall =
  (E_op == LOAD) && (E_rd != 5'd0) &&
  ( (is_D_use_rs1 && (D_rs1 == E_rd)) ||
    (is_D_use_rs2 && (D_rs2 == E_rd)) );
```

(C) Control Hazard / Flush

Trigger Condition:

Control hazard 發生於 **branch 或 jump 指令**，因為下一個 PC 直到 EX stage 才能確定。當 EX stage 判斷出「需要跳轉」時，pipeline 中 IF/ID（以及可能的 ID/EX）已經取到錯誤路徑（wrong-path）的指令，必須將其 flush。

在本設計中，當以下條件成立時，會觸發 PC redirect 與 pipeline flush：

- $E_op == \text{JAL}$ （無條件跳轉）
- $E_op == \text{JALR}$ （無條件跳轉）
- $E_op == \text{BRANCH}$ 且分支條件成立 ($alu_out == 1$)

對應程式碼：

```
if (E_op == JAL) begin
  next_pc_sel = 1'b1;
end else if (E_op == JALR) begin
  next_pc_sel = 1'b1;
end else if (E_op == BRANCH) begin
  next_pc_sel = (alu_out == 1'b1);
end else begin
  next_pc_sel = 1'b0;
end
```

一旦 $next_pc_sel == 1$ ，表示發生 control hazard，錯誤路徑的指令會在：

- Reg_D 中被替換為 NOP
- Reg_E 插入 bubble 以確保錯路指令不會影響後續執行。

2. Resolution Strategy

本實驗採用經典 5-stage pipeline (IF/ID/EX/MEM/WB)，透過 **資料前遞 (forwarding)**、**插入 bubble (stall)**、以及 **flush 錯路指令 (control flush)** 來處理 hazards。以下依序說明三種 hazard 的解法，並直接對應程式碼中的控制訊號與 pipeline register 行為。

(A) Forwarding (資料前遞) 的解決策略

策略概念： 當 EX stage 需要的 operand (**rs1 / rs2**) 在 regfile 中仍是舊值時，本設計不等待寫回，而是從 **更後面的 pipeline stage** 把最新結果直接送回 EX 使用，避免 stall。

本設計做了兩層 forwarding：

1. EX stage forwarding (主要)

- 若資料在 **M stage (且不是 LOAD)**，優先從 M stage 的 ALU result forward
- 否則若資料已在 **W stage**，就從 W stage 的 writeback data forward
- 若都不符合則使用 Reg_E 中原始 **rs*_data_out**

2. ID stage forwarding (輔助)

- ID stage 讀 regfile 時，若該值剛好在 W stage 要寫回，則直接使用 W stage 的 **wb_data** (避免 D stage 讀到舊值)

EX stage forwarding 的實作行為 (對應 mux 選擇)

本設計透過 **E_rs1_data_sel**、**E_rs2_data_sel** 控制 MUX3 選擇來源。MUX3 的輸入對應為 (以 rs1 為例)：

- **sel=1**：選 **M stage ALU out** (in1)
- **sel=2**：選 **W stage wb_data** (in0)
- **sel=0**：選 **原始 rs1_data (Reg_E)** (in2)

對應程式碼 (priority: M > W > original)：

```
if (is_E_rs1_M_overlap)      E_rs1_data_sel = 2'd1;
else if (is_E_rs1_W_overlap) E_rs1_data_sel = 2'd2;
else                         E_rs1_data_sel = 2'd0;

if (is_E_rs2_M_overlap)      E_rs2_data_sel = 2'd1;
else if (is_E_rs2_W_overlap) E_rs2_data_sel = 2'd2;
else                         E_rs2_data_sel = 2'd0;
```

ID stage forwarding 的實作行為

在 D stage 若發現 **D_rs1 == W_rd** (或 **D_rs2 == W_rd**)，則讓 D 的 mux 直接選 W stage 的 **wb_data**：

```
D_rs1_data_sel = (is_D_rs1_W_overlap) ? 1'b0 : 1'b1;
D_rs2_data_sel = (is_D_rs2_W_overlap) ? 1'b0 : 1'b1;
```

總結： Forwarding 的解決策略是「以 mux 改選資料來源」，讓 EX（與部分 D）直接拿到最新值，避免 pipeline 停住。

(B) Load-Use Stall的解決策略

策略概念： 當 EX stage 是 **LOAD**，下一條（D stage）立刻需要使用 **E_{rd}** 的值時，即使 forwarding 也無法提供正確資料（因為 load data 要到 W 才 ready），因此必須：

1. **stall 1 cycle**：凍結 PC 與 IF/ID（讓 D 那條指令不要前進）
2. **insert bubble**：將 ID/EX（或 EX stage）注入 NOP/bubble，空轉一拍
3. 下一 cycle 後，load 的資料進入 W stage，就能 forward 或寫回，D 那條指令再繼續前進

實作行為（凍結哪些、清哪些）

- **stall=1** 時：
 - **Reg_D**：hold（凍結），保留原本的 **pc_out/inst_out**
 - **Reg_E**：插入 bubble（清成 0 + bubble=1），避免錯誤執行
 - 之後 load 的結果到 W stage，pipeline 恢復正常

對應程式碼：

1) Stall 產生

```
stall = is_load_use_stall;
```

2) IF/ID 凍結 (Reg_D stall hold)

```
else if (stall) begin
    pc_out      <= pc_out;
    inst_out    <= inst_out;
    bubble_out  <= bubble_out;
end
```

3) ID/EX 插入 bubble (Reg_E 在 stall 時清空)

```
else if (stall) begin
    pc_out          <= 32'd0;
    rs1_data_out    <= 32'd0;
    rs2_data_out    <= 32'd0;
    sext_imm_out    <= 32'd0;
    bubble_out      <= 1'b1;
end
```

總結： Load-use 的解法不是 forwarding，而是「**freeze D (hold) + bubble EX (NOP)**」停一拍等資料 ready。

(C) Control Flush 的解決策略

策略概念： 當 EX stage 判斷出 branch taken 或 jump (JAL/JALR) 時，代表 IF/ID 中已經 fetch 到錯路指令。本設計採用 **redirect PC + flush**：

1. EX 得到跳轉成立 → **next_pc_sel = 1** (PC 改走 target)
2. 同時將 pipeline 中錯路指令 **flush 成 NOP / bubble**，確保它們不會 commit

實作行為

- 當 **next_pc_sel==1** (redirect 發生)：
 - **Reg_D**：把 D stage 指令 **換成 NOP** (flush wrong-path in ID)
 - **Reg_E**：把進入 EX 的資料 **清成 bubble** (避免錯路指令在 EX 執行)
 - **Controller** 的 D→E instruction tracking 也會注入 NOP (保持控制訊號一致)

對應程式碼：

1) Redirect PC (在 EX 判斷)

```
if (E_op == JAL)      next_pc_sel = 1'b1;
else if (E_op == JALR)next_pc_sel = 1'b1;
else if (E_op == BRANCH) next_pc_sel = (alu_out == 1'b1);
else next_pc_sel = 1'b0;
```

2) Flush IF/ID (Reg_D 在 jb 時注入 NOP)

```
if (jb) begin
  inst_out  <= NOP_INST;
  bubble_out <= 1'b1;
end
```

3) Flush / bubble ID/EX (Reg_E 在 jb 時清空)

```
if (jb) begin
  pc_out      <= 32'd0;
  rs1_data_out <= 32'd0;
  rs2_data_out <= 32'd0;
  sext_imm_out <= 32'd0;
  bubble_out  <= 1'b1;
end
```

4) Controller 的 D→E tracking 在 redirect 時也注入 NOP

```
if (next_pc_sel) begin
  E_op <= OP_IMM; E_f3 <= 3'b000; E_f7 <= 1'b0;
  E_rd <= 5'd0; E_rs1 <= 5'd0; E_rs2 <= 5'd0;
end
```

總結： Control hazard 的解法是「EX 決定跳轉後，**redirect PC + flush D/E (NOP/bubble)**」，清掉 wrong-path 指令。

3. Implementation Logic (在 **Controller.sv** 的實作邏輯)

本實驗在 **Controller.sv** 中負責產生 pipeline 控制信號 (forwarding select、stall、next_pc_sel)，並以 pipeline tracking registers (D→E→M→W 的 *_op/f3/f7/rd/rs*) 來做 hazard 偵測。對應的 pipeline registers (**Reg_D/Reg_E/...**) 依據 **stall** 與 **jb(next_pc_sel)** 執行 hold / bubble / flush。

(A) Forwarding：如何選擇資料來源？ (含 priority)

1) ID stage: W → D 的 immediate forwarding

****目的：** **避免 D stage 讀 regfile 讀到舊值。 ****作法：** **當 D 的 **rs1/rs2** 剛好等於 W stage 的 **rd** (且 W 會寫回、且 **rd != x0**) 時，D stage 直接改選 **wb_data**。

- 判斷訊號: **is_D_rs1_W_overlap, is_D_rs2_W_overlap**
- 控制訊號: **D_rs1_data_sel, D_rs2_data_sel**
- 注意: **MUX2** 編碼是 **sel=1 選 in0, sel=0 選 in1**，因此「選 **wb_data**」時要給 **sel=0**

對應程式碼：

```
is_D_rs1_W_overlap = is_D_use_rs1 && is_W_use_rd &&
                      (D_rs1 != 5'd0) && (D_rs1 == W_rd);
is_D_rs2_W_overlap = is_D_use_rs2 && is_W_use_rd &&
                      (D_rs2 != 5'd0) && (D_rs2 == W_rd);

// select WB => sel=0 ; select normal => sel=1
D_rs1_data_sel = (is_D_rs1_W_overlap) ? 1'b0 : 1'b1;
D_rs2_data_sel = (is_D_rs2_W_overlap) ? 1'b0 : 1'b1;
```

2) EX stage: M/W → E forwarding (priority: M > W > original)

****目的：** **EX stage 需要的 operand 若尚未寫回 regfile，直接從後面 stage forward。 ****作法：** **用 **MUX3** 由 **E_rs*_data_sel** 控制三選一：

- sel=1**：選 M stage ALU result (in1)
- sel=2**：選 W stage **wb_data** (in0)

- `sel=0`: 選 Reg_E 原始 `rs*_data` (in2)

Priority 規則:

1. 若 M stage 可 forward (且 `M_op != LOAD`) → 優先 M
2. 否則若 W stage match → 選 W
3. 否則不 forward

對應程式碼:

```
// M forwarding (only if M is NOT LOAD)
is_E_rs1_M_overlap = is_E_use_rs1 && is_M_use_rd &&
    (E_rs1 != 5'd0) && (E_rs1 == M_rd) &&
    (M_op != LOAD);
is_E_rs2_M_overlap = is_E_use_rs2 && is_M_use_rd &&
    (E_rs2 != 5'd0) && (E_rs2 == M_rd) &&
    (M_op != LOAD);

// W forwarding
is_E_rs1_W_overlap = is_E_use_rs1 && is_W_use_rd &&
    (E_rs1 != 5'd0) && (E_rs1 == W_rd);
is_E_rs2_W_overlap = is_E_use_rs2 && is_W_use_rd &&
    (E_rs2 != 5'd0) && (E_rs2 == W_rd);

// Priority: M > W > original
if (is_E_rs1_M_overlap)    E_rs1_data_sel = 2'd1;
else if (is_E_rs1_W_overlap) E_rs1_data_sel = 2'd2;
else                      E_rs1_data_sel = 2'd0;

if (is_E_rs2_M_overlap)    E_rs2_data_sel = 2'd1;
else if (is_E_rs2_W_overlap) E_rs2_data_sel = 2'd2;
else                      E_rs2_data_sel = 2'd0;
```

(B) Stalls: 哪些 pipeline registers freeze? 哪些被 flush?

1) stall 產生條件 (load-use)

本設計用 `is_load_use_stall` 偵測「E 是 LOAD 且 D 需要 `E_rd`」:

```
is_load_use_stall =
    (E_op == LOAD) && (E_rd != 5'd0) &&
    ( (is_D_use_rs1 && (D_rs1 == E_rd)) ||
      (is_D_use_rs2 && (D_rs2 == E_rd)) );

stall = is_load_use_stall;
```

2) stall 時 pipeline 行為 (freeze D、bubble EX)

策略：

- **Reg_D (IF/ID) hold**: D 指令不前進 (相當於 freeze PC 與 IF/ID)
- **Reg_E (ID/EX) 插入 bubble**: 送一個 NOP/bubble 進 EX, 空轉一拍
- M/W 照常流動 (讓 load 繼續往下走到 W 產生資料)

對應 **Reg_D.sv** (stall 時 hold) :

```
else if (stall) begin
    pc_out      <= pc_out;
    inst_out    <= inst_out;
    bubble_out  <= bubble_out;
end
```

對應 **Reg_E.sv** (stall 時清空成 bubble) :

```
else if (stall) begin
    pc_out      <= 32'd0;
    rs1_data_out <= 32'd0;
    rs2_data_out <= 32'd0;
    sext_imm_out <= 32'd0;
    bubble_out  <= 1'b1;
end
```

此外 **Controller** 的 D→E instruction tracking 也會在 stall 時注入 NOP, 確保 control signals 不會誤觸發:

```
else if (stall) begin
    E_op  <= OP_IMM; E_f3 <= 3'b000; E_f7 <= 1'b0;
    E_rd  <= 5'd0; E_rs1 <= 5'd0; E_rs2 <= 5'd0;
end
```

(C) Branch/Jump: 如何清 wrong-path? (flush 機制與 priority)

1) EX 判斷 redirect (next_pc_sel)

本設計在 EX stage 產生 **next_pc_sel**, 表示下一個 PC 要跳到 target:

```
if (E_op == JAL) begin
    next_pc_sel = 1'b1;
end else if (E_op == JALR) begin
    next_pc_sel = 1'b1;
end else if (E_op == BRANCH) begin
    next_pc_sel = (alu_out == 1'b1);
end else begin
```

```
next_pc_sel = 1'b0;
end
```

補充: BRANCH 的比較 operand2 必須選 rs2, 因此 Controller 需讓 `E_alu_op2_sel=1` (`sel=1→rs2`) 在 `E_op==BRANCH` 時成立, 避免用 imm 去比較造成分支判斷錯誤。

2) flush 範圍: Reg_D flush + Reg_E bubble

當發生 redirect (`jb=next_pc_sel=1`) 代表 IF/ID 取到錯路指令, 必須清除:

- **Reg_D**: 直接把 D 的 instruction 置換為 NOP (flush)
- **Reg_E**: 將進入 EX 的內容清空、bubble=1 (避免錯路在 EX 執行)
- **Controller** 的 D→E tracking 同步注入 NOP (保持控制一致)

對應 `Reg_D.sv` (`jb` 時 flush NOP) :

```
if (jb) begin
    inst_out    <= NOP_INST;
    bubble_out  <= 1'b1;
end
```

對應 `Reg_E.sv` (`jb` 時 bubble) :

```
if (jb) begin
    pc_out        <= 32'd0;
    rs1_data_out  <= 32'd0;
    rs2_data_out  <= 32'd0;
    sext_imm_out  <= 32'd0;
    bubble_out    <= 1'b1;
end
```

對應 `Controller.sv` (redirect 時 D→E 注入 NOP) :

```
if (next_pc_sel) begin
    E_op  <= OP_IMM; E_f3 <= 3'b000; E_f7 <= 1'b0;
    E_rd  <= 5'd0; E_rs1 <= 5'd0; E_rs2 <= 5'd0;
end
```

(D) Priority: redirect (flush) 與 stall (bubble) 誰先?

本設計採用 **redirect 優先於 stall** (因為一旦確定跳轉, 錯路指令必須先清掉)。因此在 **Controller** 的 D→E tracking 更新順序上是:

```

if (next_pc_sel) begin
    // flush NOP
end else if (stall) begin
    // bubble NOP
end else begin
    // normal D->E latch
end

```

4. Branch Prediction

(A) 我會選哪種 Branch Predictor? 為什麼?

選擇: **Dynamic 2-bit Saturating Counter+ BTB (Branch Target Buffer)** 原因:

1. **qsort/迴圈類程式分支規律強**: 迴圈分支多為「大多 taken, 最後一次 not taken」, 2-bit 比 1-bit 更能避免「迴圈結束那次」造成的反覆抖動。
2. **2-bit 的硬體成本低, 但效益大**: 只要每個 entry 2 bits state (00/01/10/11), 就能顯著降低 mispredict。
3. **搭 BTB 才能真正提速**: 只有預測方向 (taken/not taken) 還不夠, taken 時還需要立即知道 target PC 才能讓 IF 不停。

(B) 若發生 Misprediction, 我會怎麼設計控制信號來解 control hazard?

1) Baseline —— 「Always Not Taken」+ EX redirect

程式的策略是:

- IF 預設取 `pc+4`
- EX 若判斷要跳 (JAL/JALR 必跳、BRANCH taken) 就讓:
 - `next_pc_sel = 1` → PC 改走 target
 - 同時 flush IF/ID (Reg_D 注入 NOP) 與 bubble ID/EX (Reg_E 清空)

對應 `Controller.sv` (EX 決定 redirect) :

```

if (E_op == JAL) begin
    next_pc_sel = 1'b1;
end else if (E_op == JALR) begin
    next_pc_sel = 1'b1;
end else if (E_op == BRANCH) begin
    next_pc_sel = (alu_out == 1'b1);
end else begin
    next_pc_sel = 1'b0;
end

```

對應 pipeline flush (以 `jb = next_pc_sel` 為 flush 控制) :

- `Reg_D.sv`: `jb==1` → `inst_out <= NOP`
- `Reg_E.sv`: `jb==1` → 清空資料並 `bubble_out<=1`

因此在 baseline 下:

- 若 **branch/jump 實際 taken**: 一定會 mispredict (因為預設 not taken) , 需要 flush
- 若**實際 not taken**: 不需 flush

2) 若改成「有分支預測」的設計: 需要哪些新訊號?

新增以下概念性訊號:

- `pred_taken_F`: 在 IF stage 預測此指令是否 taken
- `pred_target_F`: 若預測 taken, 對應 target PC (由 BTB 提供)
- `pc_sel_F`: IF stage PC 選擇 (`pc+4` vs `pred_target_F`)
- `pred_taken_E`、`pred_target_E`: 把預測結果跟著指令一起 pipeline 到 EX, 用來檢查是否誤判
- `actual_taken_E`、`actual_target_E`: EX 真實結果 (由 `alu_out` 與 `jb target` 計算得到)
- `mispredict_E`: 誤判判定訊號

3) Mispredict 判定邏輯 (在 EX stage 做)

在 EX stage 比較「預測」與「真實」:

```
actual_taken_E = (E_op==JAL) | (E_op==JALR) | ((E_op==BRANCH) & alu_out);
actual_target_E = jb_out; // 由 EX 算出來的目標

mispredict_E =
    (pred_taken_E != actual_taken_E) ||
    (pred_taken_E && actual_taken_E && (pred_target_E != actual_target_E));
```

4) Mispredict 發生時要怎麼 flush?

- `jb_flush = mispredict_E` (而不是單純 taken)
- `next_pc_sel` (或 PC redirect) 改為:
 - 若 mispredict: PC 走 `actual_target_E` (taken) 或回到 `pc+4` (not taken)
 - 若 predict correct: 不需要 flush

flush 的作用範圍 (概念上) 仍然一致:

- **flush IF/ID (Reg_D 注入 NOP)**
- **bubble ID/EX (Reg_E 清空)**
- priority 仍為: `flush > stall`

沿用現有程式的控制結構：

```
if (jb_flush) begin
    // flush D/E => NOP/bubble
end else if (stall) begin
    // load-use bubble
end else begin
    // normal pipeline advance
end
```

(C) 2-bit Predictor 的更新策略

每個分支在 predictor table 中有 2-bit state：

- 00 Strongly Not Taken
- 01 Weakly Not Taken
- 10 Weakly Taken
- 11 Strongly Taken

在 EX 得到 `actual_taken_E` 後更新：

- taken → state 向 11 飽和遞增
- not taken → state 向 00 飽和遞減

並用 state 的 MSB 作為 `pred_taken`。

Reflection Report (0 pts)

In this section, please write a short reflection about the lab:

1. What did you learn from completing this assignment?
2. What challenges did you encounter, and how did you solve them?
3. What feedback would you like to give to the TAs regarding this lab?

Reflection Report

1. What did you learn from completing this assignment?

透過 Lab 4 的實作，我對 **五階段 Pipeline CPU 的實際運作方式** 有了更深入的理解。過去在課堂上學到的 forwarding、stall、flush 多半停留在概念層次，但在這次作業中，必須將這些概念真正落實到 **Controller 的控制訊號與 pipeline registers 的行為**，才發現許多細節非常關鍵。

(1) **Forwarding 的 priority ($M > W$)** 為什麼不能寫錯，否則會使用到較舊的資料。

(2) **Load-use hazard 無法單靠 forwarding 解決**，一定需要 stall 一個 cycle 並插入 bubble。

(3) **Branch/jump 的控制冒險本質上是 PC 選擇與 wrong-path 指令清除的問題**，flush 的範圍與優先序會直接影響程式正確性。

這次實驗讓我真正體會到，CPU pipeline 的正確性來自於「每一個控制訊號都必須在正確的 stage、正確的 cycle 發生」。

2. What challenges did you encounter, and how did you solve them?

最大的挑戰是在 **debug pipeline 行為**，尤其是程式可以成功編譯，但在跑 `qsort_test` 時卻出現 PC mismatch 或結果錯誤。這類錯誤不像語法錯誤那樣直觀，而是來自某個 cycle 的控制訊號不正確。

我遇到的主要問題包括：

- **Branch 指令的 ALU operand 選擇錯誤**（誤用 immediate 而非 rs2），導致分支判斷錯誤、PC 逐漸偏離 reference。
- **Flush 與 stall 的優先序** 必須清楚定義，否則在 branch + load-use 同時發生時會出現非預期行為。

解決方式是：

- 透過 `ctest --output-on-failure` 觀察第一個 mismatch 的 PC。
- 回頭對照 pipeline timing diagram，逐步檢查 EX stage 的控制訊號是否符合指令語意。
- 將錯誤縮小到單一類型的 hazard，再修正對應的 Controller 邏輯。

這個過程雖然花時間，但也讓我對 pipeline 的資料流與控制流理解更加扎實。

3. What feedback would you like to give to the TAs regarding this lab?

我認為這是一個**學習成效很高的實驗**，能有效訓練學生對 CPU pipeline 細節的理解。不過也有幾點建議：

- (1) **在 Lab handout 中多強調 mux select 的編碼方向** 實作時很容易因為 mux 的 sel 定義與直覺相反而出錯，若能特別提醒會很有幫助。
- (2) **qsort test 對 pipeline 正確性非常敏感** 雖然有挑戰性，但建議能搭配一個較簡單、逐步驗證 hazard 的測資，讓學生能循序確認功能正確。

整體而言，這次 Lab 具有很高的實作與學習價值，完成後對 CPU micro-architecture 的理解有明顯提升。
